

■ Cours Express — Catégorie 5

■ Core Code Anti-Patterns (Transformers, PyTorch & RAG) (Mistral FDE - PRIORITÉ #4)

1. ■ Optimisation VRAM GPU & Inférence PyTorch

A. Absence de `@torch.no\_grad()` pendant l'Inférence

Problème : Garder le graphe de calcul autograd actif pendant le décodage de tokens.

Impact Scalabilité : Explosion de l'empreinte VRAM GPU (3x à 5x plus de mémoire) → Crash `CUDA Out Of Memory` (OOM).

■ Code Buggué	■ Correction Propre
<pre>def generate_tokens(model, inputs): # ■ Conserve le graphe autograd inutilement ! return model.generate(inputs)</pre>	<pre>@torch.no_grad() # ■ Libère le graphe d'autograd pendant l'inférence def generate_tokens(model, inputs): return model.generate(inputs)</pre>

B. Re-allocation de Mémoire par `torch.cat` dans la Boucle KV Cache

Problème : Concaténer un nouveau tenseur (K, V) à chaque token généré.

Impact Scalabilité : Fragmentation de la VRAM GPU et réallocations mémoire $O(N^2)$, effondrant le débit (tokens/sec).

■ Code Buggué	■ Correction Propre
<pre>for token in range(max_len): kv_cache = torch.cat([kv_cache, new_kv], dim=1) # ■ Allocations $O(N^2)$</pre>	<pre># ■ Pré-allocation du KV Cache à max_seq_len kv_cache = torch.zeros((batch, heads, max_len, head_dim)) kv_cache[:, :, current_pos] = new_kv</pre>

2. ■ Core Mechanics & Incompatibilités PyTorch

A. Incompatibilité de Device PyTorch (`CUDA` vs `CPU`)

Problème : Multiplication matricielle entre un tenseur sur GPU (`cuda:0`) et un tenseur sur CPU.

Impact Exécution : Crash serveur immédiat RuntimeError: Expected all tensors to be on the same device.

■ Code Buggué	■ Correction Propre
<pre>inputs = tokenizer(text, return_tensors='pt') # CPU par défaut outputs = model(inputs['input_ids']) # ■ model est sur GPU (cuda:0) !</pre>	<pre>inputs = tokenizer(text, return_tensors='pt').to(device) outputs = model(inputs['input_ids']) # ■ Même device</pre>

B. Division par Zéro dans le Cosine Similarity (Vector Search)

Problème : Calculer `dot(A, B) / (norm(A) \* norm(B))` sans valeur d'épsilon.

Impact Scalabilité : Produit des `NaN` qui se propagent et corrompent tout le pipeline RAG/recherche.

■ Code Buggué	■ Correction Propre
<pre>def cosine_sim(a, b): return torch.dot(a, b) / (torch.norm(a) * torch.norm(b)) # ■ NaN si 0</pre>	<pre>def cosine_sim(a, b, eps=1e-8): denom = torch.clamp(torch.norm(a) * torch.norm(b), min=eps) return torch.dot(a, b) / denom # ■ Protégé avec epsilon</pre>

1. 'Activer `@torch.no_grad()` et `model.eval()`' → Obligatoire avant toute inférence pour éviter les OOM VRAM et désactiver le Dropout.
2. 'Pré-allouer le KV Cache' → Bannir `torch.cat` dans la boucle autorégressive pour maintenir un débit élevé.
3. 'Vérifier l'alignement des Devices CUDA/CPU' → Toujours faire `tensor.to(device)` avant multiplication matricielle.
4. 'Sécuriser le Cosine Similarity avec ϵ ' → Utiliser `torch.clamp(..., min=1e-8)` contre les division par zéro.